

TRABAJO FIN DE MÁSTER
MÁSTER INTERUNIVERSITARIO EN
COMPUTACIÓN DE ALTAS PRESTACIONES

Weather Forecasting Using Deep Learning Models

Student: Olivier Tamon
Supervisor: Guillermo López Taboada
A Coruña, June 21, 2026.

To Mr. Metzger, my mathematics teacher, who helped shape my thinking and analytical skills.

Acknowledgements

I would like to express my sincere gratitude to all my professors throughout my academic journey, and in particular to Juan Angel Lorenzo del Castillo, Karima, and my master's thesis supervisor, Guillermo López Taboada, for their guidance and support.

I would also like to thank Léo De sario, Yvan Moalic, and Cédric Hillion for being by my side during the years in which I built who I am.

I am deeply grateful to Gauthier Dewolf, Yu Tsukimura, Mathys Rey and Tristan Noterman for standing by me through the mentally hardest years of my life.

I am especially thankful to Thomas Brasdefer, Enzo Fernandez, and Antoine Kraus for accompanying me throughout these three years of study.

I would also like to thank Thomas Llasera for encouraging open-mindedness and providing valuable perspective.

A special thank you goes to Clément Cros, who came with me to Spain and shared the experience of the double degree program by my side.

Finally, I am deeply grateful to my parents for always believing in me and for their unwavering support.

Abstract

Weather forecasting is essential for many societal and economic activities, including agriculture, transportation, disaster prevention, and energy management. Traditional forecasting systems rely primarily on Numerical Weather Prediction (NWP), which uses physical models to simulate atmospheric dynamics. Although these models provide highly accurate predictions, they require considerable computational resources and specialized infrastructure. Recent advances in artificial intelligence and machine learning have opened new possibilities for data-driven weather forecasting. Deep learning models are capable of learning complex spatial and temporal relationships directly from large meteorological datasets. This project investigates the application of deep learning methods for short-term weather forecasting using the ERA5 reanalysis dataset. Several neural network architectures will be explored, including convolutional neural networks and encoder–decoder models such as U-Net. The models are trained using PyTorch and evaluated using quantitative metrics and visualization techniques. The objective of this work is to assess the ability of deep learning models to capture atmospheric patterns and generate reliable weather predictions.

Keywords:

- Deep Learning
- Numerical Weather Prediction
- ERA5 Reanalysis
- Convolutional Neural Networks
- Encoder–Decoder Architecture
- Temperature Forecasting
- Spatio-Temporal Modeling

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	1
2	State of the Art	3
2.1	Numerical Weather Prediction	3
2.2	Limitations of Numerical Weather Prediction	3
2.3	Statistical Forecasting Methods	4
2.4	Machine Learning in Weather Forecasting	4
2.5	Deep Learning for Meteorological Data	4
2.5.1	Convolutional Neural Networks	4
2.5.2	Recurrent Neural Networks	5
2.5.3	Spatio-Temporal Neural Networks	5
2.5.4	Encoder–Decoder Architectures	5
2.5.5	Large-Scale AI Weather Forecasting Systems	5
3	Dataset and Data Preparation	7
3.1	ERA5 Dataset Description	7
3.2	Data Acquisition	8
3.2.1	Data Collection	8
3.2.2	Dataset Temporal Split	9
3.3	Data Format	9
4	Methodology	11
4.1	Weather Forecasting Pipeline	11
4.1.1	Data acquisition	11

4.1.2	Data preprocessing	11
4.1.3	Model training	12
4.1.4	Forecast generation	12
4.1.5	Evaluation and visualization	12
4.2	Data Preprocessing	12
4.3	Temporal Windowing	13
4.4	Neural Network Architectures	13
4.4.1	Convolutional Neural Networks (CNNs)	14
4.4.2	Encoder–decoder networks (U-Net)	14
4.4.3	Recurrent Neural Networks	14
4.4.4	Hybrid spatio-temporal models	14
5	Implementation	15
5.1	Software Environment	15
5.1.1	Python	15
5.1.2	PyTorch	15
5.1.3	Numpy	15
5.1.4	Xarray	16
5.1.5	Matplotlib	16
5.1.6	Additional libraries	16
5.2	Hardware Environment	16
5.2.1	Google Colab GPU environment	16
5.2.2	Local workstation	17
5.2.3	HPC supercomputer environment	17
5.2.4	Hardware configuration summary	17
6	Experimental Results	19
6.1	Evaluation Metrics	19
6.1.1	Mean Squared Error (MSE)	19
6.1.2	Root Mean Squared Error (RMSE)	19
6.1.3	Mean Absolute Error (MAE)	20
6.2	Visualization	20
6.2.1	Training process	20
6.2.2	Example of temperature prediction	21
6.3	Forecast accuracy over multiple days	21
7	Discussion	23
8	Conclusion	25

CONTENTS

A	Glosario de acrónimos	29
B	Glosario de terminos	31
	Bibliography	33

List of Figures

6.1	Learning curves of the U-Net model trained using the Mean Squared Error (MSE) loss function.	20
6.2	Spatial field comparison between ground truth (left) and model prediction (right) at lead time $t+6$	21
6.3	Evolution of RMSE and MAE as a function of forecast lead time.	22

List of Tables

3.1	Temporal split of the dataset	9
-----	---	---

Introduction

1.1 Background

Weather forecasting plays a crucial role in many areas such as disaster prevention, aviation, agriculture, and renewable energy management. Accurate weather predictions allow governments and industries to anticipate extreme events and improve decision-making processes.

Modern forecasting systems rely mainly on Numerical Weather Prediction (NWP) models. These systems simulate atmospheric behavior using mathematical equations describing fluid dynamics and thermodynamics.

However, running numerical weather models requires powerful supercomputers and complex infrastructure. As a result, researchers have started exploring machine learning approaches capable of learning atmospheric dynamics directly from historical meteorological data.

1.2 Problem Statement

Traditional numerical weather models are computationally expensive and require significant expertise to operate. At the same time, the increasing availability of large meteorological datasets provides an opportunity to develop data-driven forecasting models. The key question addressed in this project is: Can deep learning models learn atmospheric patterns from historical data and produce accurate short-term weather forecasts?

1.3 Objectives

The objective of this project is to investigate the application of machine learning techniques to weather forecasting. This includes reviewing current research in the field, collecting and

preprocessing meteorological data from the ERA5 dataset, and designing and implementing deep learning models for weather prediction. In addition, the study aims to compare different neural network architectures and evaluate their forecasting performance using appropriate quantitative metrics.

State of the Art

Weather forecasting has evolved significantly over the past decades with the development of advanced numerical models and increased computational power. Recently, artificial intelligence techniques have emerged as promising alternatives for modeling atmospheric dynamics. This chapter reviews existing research in weather forecasting, including traditional numerical approaches and modern machine learning methods.

2.1 Numerical Weather Prediction

Numerical Weather Prediction (NWP) models simulate atmospheric dynamics by solving physical equations governing fluid motion and thermodynamics. These models divide the atmosphere into three-dimensional grids and compute the evolution of meteorological variables over time. Operational forecasting centers such as the European Centre for Medium-Range Weather Forecasts (ECMWF) operate global models capable of producing forecasts several days in advance. Key predicted variables include temperature, pressure, wind speed, humidity, and geopotential height. Although these models provide highly accurate forecasts, they require substantial computational resources.

2.2 Limitations of Numerical Weather Prediction

Despite their accuracy, Numerical Weather Prediction (NWP) systems present several important limitations. First, they involve a high computational cost, as global simulations require powerful supercomputers. In addition, these models rely on complex parameterization schemes, where many atmospheric processes must be approximated using simplified representations. NWP systems are also highly sensitive to initial conditions: even small errors in the initial state of the atmosphere can lead to significant prediction inaccuracies due to

the chaotic nature of atmospheric dynamics. Furthermore, despite the availability of large volumes of atmospheric and observational data, traditional NWP models do not fully exploit these datasets. Much of the available data remains underutilized, as these systems primarily rely on physically based models and specific data assimilation methods rather than learning directly from historical data. These limitations motivate the exploration of data-driven forecasting approaches.

2.3 Statistical Forecasting Methods

Before the rise of machine learning, statistical models were widely used to forecast meteorological variables. Common approaches included linear regression models, autoregressive models, ARIMA models [1], and Kalman filtering techniques. These methods aim to capture relationships between past and future atmospheric states based on statistical correlations. While they can be effective for simple time-series forecasting tasks, they generally struggle to model the complex nonlinear dynamics inherent to atmospheric systems.

2.4 Machine Learning in Weather Forecasting

Machine learning methods provide an alternative approach to forecasting by learning patterns directly from historical data. Early applications of machine learning in meteorology relied on algorithms such as decision trees, support vector machines, and shallow neural networks. These techniques have been used for precipitation prediction, temperature forecasting [2], and storm classification. However, traditional machine learning models are limited in their ability to process high-dimensional spatial data.

2.5 Deep Learning for Meteorological Data

Deep learning models have significantly improved the ability to process large meteorological datasets. Meteorological data is naturally represented as spatio-temporal grids, which makes deep learning architectures such as convolutional neural networks particularly suitable. Deep learning models can capture complex relationships between atmospheric variables and learn hierarchical representations of spatial patterns.

2.5.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are widely used for spatial data analysis. These networks apply convolutional filters to detect spatial patterns in input data. In meteorology, CNNs are capable of identifying important structures such as pressure systems, temperature

gradients, and cloud formations [3]. CNN-based models have been successfully applied to tasks such as precipitation forecasting and radar image analysis.

2.5.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are designed to model temporal sequences. Variants such as Long Short-Term Memory (LSTM) networks are commonly used for time series forecasting tasks. RNNs can capture temporal dependencies between atmospheric states and have been used to predict variables such as temperature and rainfall [4].

2.5.3 Spatio-Temporal Neural Networks

To combine spatial and temporal modeling, researchers have proposed hybrid architectures that integrate convolutional and recurrent layers. One example is the ConvLSTM [5] architecture, which replaces fully connected operations in LSTM networks with convolutional layers. These models are widely used for precipitation nowcasting and radar image forecasting.

2.5.4 Encoder–Decoder Architectures

Encoder–decoder networks are commonly used for spatial prediction tasks. These networks encode input data into feature representations and then decode them to produce output predictions. One example is U-Net, which introduces skip connections between encoder and decoder layers to preserve spatial details [6]. In this project, encoder–decoder models may be tested among other architectures.

2.5.5 Large-Scale AI Weather Forecasting Systems

Recent advances in artificial intelligence have produced large-scale weather forecasting models trained on decades of meteorological data. Examples include:

- **GraphCast**

GraphCast is a machine learning model developed by DeepMind that predicts global weather conditions using graph neural networks. It was trained on ERA5 data and has demonstrated strong forecasting performance [7].

- **Pangu-Weather**

Pangu-Weather is a transformer-based forecasting model capable of generating high-resolution global forecasts [8].

- **FourCastNet**

FourCastNet uses Fourier neural operators to model atmospheric dynamics and generate global forecasts efficiently [9].

Dataset and Data Preparation

3.1 ERA5 Dataset Description

The dataset used in this project is derived from the ERA5 reanalysis dataset, produced by the European Centre for Medium-Range Weather Forecasts (ECMWF). ERA5 is a global atmospheric reanalysis that combines numerical weather prediction models with observational data from satellites, weather stations, aircraft, and other measurement systems to reconstruct past atmospheric states.

ERA5 provides a comprehensive representation of the Earth's atmosphere with hourly temporal resolution and global spatial coverage. The dataset includes a large number of atmospheric variables such as temperature, wind speed, humidity, pressure, and geopotential height, available at multiple pressure levels and spatial resolutions.

Although ERA5 contains a wide variety of meteorological variables, this project adopts a simplified configuration in order to validate the deep learning pipeline and reduce computational complexity during the experimentation phase. More specifically, the experiments conducted in this work focus on a single variable: air temperature at the 1000 hPa pressure level. This variable represents the atmospheric temperature close to the Earth's surface and is commonly used in meteorological forecasting studies.

The choice of using a single variable facilitates the development and evaluation of the forecasting models while keeping data preprocessing and training processes manageable. In this study, surface temperature is therefore used as the sole input variable and is measured in Kelvin (K). Although the ERA5 dataset provides many additional meteorological variables, such as wind components, humidity, and geopotential height, these are not included in the current experiments in order to simplify the modeling process and validate the forecasting

pipeline.

Nevertheless, it is important to note that incorporating additional atmospheric variables could potentially improve forecasting accuracy in future work, as ERA5 offers a rich and diverse set of meteorological information.

3.2 Data Acquisition

3.2.1 Data Collection

The meteorological data used in this project is downloaded from the Climate Data Store (CDS) using the CDS API, which allows automated access to ERA5 datasets through Python scripts.

The request sent to the API specifies several parameters including the atmospheric variable, the pressure level, the temporal range, the geographical region, and the spatial grid resolution. The request structure used in this project is shown below:

```
request = {
    "product_type": ["reanalysis"],
    "variable": [CFG.variable],
    "pressure_level": [CFG.pressure_level],
    "year": [year],
    "month": [month],
    "day": [f"{d:02d}" for d in range(1, 32)],
    "time": list(CFG.times),
    "data_format": "netcdf",
    "download_format": "unarchived",
    "area": list(CFG.area),
    "grid": list(CFG.grid),
}
```

This request specifies the following parameters:

- **product_type** – selects reanalysis data from the ERA5 dataset
- **variable** – atmospheric variable to retrieve (temperature in this project)
- **pressure_level** – pressure level corresponding to the surface level used for the experiments
- **year, month, day** – temporal coverage of the requested data

- **time** – list of hourly timestamps included in the dataset
- **area** – geographical region selected for the study
- **grid** – spatial resolution of the dataset
- **data_format** – output format used to store the data (NetCDF)

3.2.2 Dataset Temporal Split

To train and evaluate the machine learning models, the collected data is divided into three subsets:

Dataset	Years	Purpose
Training dataset	2011–2020	Model training
Test dataset	2021	Model evaluation
Validation dataset	2022	Hyperparameter tuning and model selection

Table 3.1: Temporal split of the dataset

The training dataset is used to learn the model parameters. The validation dataset is used during training to monitor model performance and tune hyperparameters. Finally, the test dataset is used to evaluate the final performance of the trained models.

3.3 Data Format

The ERA5 data is stored in NetCDF (Network Common Data Form) format. NetCDF is a widely used data format in atmospheric and climate sciences for storing multi-dimensional scientific datasets. NetCDF files allow efficient storage and retrieval of large datasets and include meta-data describing variables, units, and dimensions. In the ERA5 dataset, each meteorological variable is represented as a multi-dimensional array indexed by spatial and temporal coordinates. Typical dimensions in the NetCDF files include:

- time – represents hourly timestamps
- latitude – spatial coordinates along the north–south axis
- longitude – spatial coordinates along the east–west axis
- variable values – meteorological measurements at each grid point

For example, the temperature variable can be represented as a three-dimensional array: temperature(time, latitude, longitude) This structure makes NetCDF particularly suitable for machine learning models that process spatio-temporal data.

Methodology

4.1 Weather Forecasting Pipeline

The weather forecasting pipeline implemented in this project follows a structured workflow composed of several stages. The goal is to transform raw atmospheric reanalysis data into machine learning predictions capable of forecasting future weather states.

The main stages of the pipeline are the following:

4.1.1 Data acquisition

Weather data are obtained from the ERA5 reanalysis dataset, provided by the Copernicus Climate Data Store (CDS). ERA5 provides high-resolution atmospheric variables reconstructed using numerical weather prediction models and observational data. In this project, the data are downloaded automatically through the CDS API using Python scripts. The selected dataset corresponds to the ERA5 pressure-level data, from which the temperature variable at the 1000 hPa pressure level is extracted over the European region. Data are downloaded monthly and stored as NetCDF files.

4.1.2 Data preprocessing

Once downloaded, the NetCDF files are loaded using the xarray library, which is designed for handling multidimensional scientific data. The temperature field is extracted, converted into NumPy arrays, and reshaped into a format compatible with deep learning frameworks. Each sample corresponds to a spatial grid of temperature values at a given time step.

4.1.3 Model training

A neural network model based on a U-Net encoder–decoder architecture is trained to learn the mapping between the atmospheric state at time t and the state at time $t+6$ hours. The dataset is divided into training, validation, and test subsets using a temporal split. During training, the model parameters are optimized using the Adam optimizer with a Mean Squared Error (MSE) loss function.

4.1.4 Forecast generation

Once trained, the best-performing model (based on validation loss) is used to generate predictions. Multi-step forecasts are produced using a recursive forecasting strategy, where the predicted state at time $t+6$ becomes the input for predicting the state at $t+12$, and so on. This allows the model to generate forecasts for several days ahead.

4.1.5 Evaluation and visualization

The quality of the predictions is evaluated using common regression metrics such as Root Mean Square Error (RMSE) and Mean Absolute Error (MAE). These metrics are computed for different forecast horizons. Results are visualized through graphs showing how prediction accuracy evolves as the forecast horizon increases.

4.2 Data Preprocessing

Data preprocessing is an essential step for transforming raw meteorological data into a format suitable for machine learning models. In this project, the preprocessing pipeline involves several operations, including data loading, variable selection, normalization, and dataset splitting.

The ERA5 data used in this study are stored in NetCDF format, which is widely employed for climate and atmospheric datasets. These files are loaded using the Python library *xarray*, which enables efficient manipulation of multidimensional arrays representing both spatial and temporal information. From each NetCDF file, the temperature variable at the 1000 hPa pressure level is extracted over the selected geographical region in Europe.

After extraction, the data are converted into NumPy arrays and reshaped into a format compatible with deep learning models. The resulting data tensor follows a four-dimensional structure defined by time, channels, height, and width, where time represents the sequence of time steps, channels corresponds to the number of atmospheric variables used as input, and height and width represent the spatial dimensions of the temperature grid. In this project, only one

variable is considered, meaning that the number of channels is equal to one.

To improve the stability and efficiency of neural network training, the data are normalized using z-score normalization. This transformation consists of subtracting the mean of the dataset from each value and dividing the result by the standard deviation. As a result, the data are centered around zero and scaled so that the variance is approximately equal to one. This normalization process facilitates faster convergence during training and enhances numerical stability.

Finally, the dataset is divided chronologically into three subsets: a training set representing 84% of the data, a validation set accounting for 8%, and a test set also representing 8%. This temporal split ensures that the model is evaluated on future data that were not seen during training, thereby providing a more realistic assessment of forecasting performance.

4.3 Temporal Windowing

Weather forecasting inherently involves temporal dependencies, as atmospheric states evolve continuously over time. Capturing these temporal relationships is therefore essential for producing accurate predictions. In this project, a time-step prediction approach is adopted, where the model is trained to predict the atmospheric state at time $t + 6$ hours from the state at time t .

During training, each sample consists of an input corresponding to the atmospheric state at time t and a target corresponding to the state at time $t + 6$, with each time step representing a 6-hour interval in the ERA5 dataset. This formulation allows the model to learn the temporal evolution of atmospheric conditions over short time horizons.

Once trained, the model is able to generate longer forecasts using a recursive prediction strategy. In this approach, the predicted output at one time step is reused as the input for the subsequent prediction. As a result, the model can iteratively produce forecasts at $t + 6$, $t + 12$, $t + 18$ hours, and beyond, enabling predictions over several days despite being trained on short-term dependencies.

4.4 Neural Network Architectures

Several deep learning architectures can be applied to the problem of weather forecasting, as atmospheric dynamics involve complex spatial and temporal interactions. Possible architectures include:

4.4.1 Convolutional Neural Networks (CNNs)

CNNs are widely used for spatial data processing. They are particularly effective for capturing local spatial structures, such as temperature gradients or pressure patterns across geographic grids.

4.4.2 Encoder–decoder networks (U-Net)

In this project, the main architecture used is a U-Net model, which follows an encoder–decoder structure. The encoder compresses the spatial information into higher-level feature representations, while the decoder reconstructs the predicted temperature field. Skip connections between encoder and decoder layers help preserve spatial details and improve prediction accuracy.

4.4.3 Recurrent Neural Networks

Recurrent architectures such as LSTM or GRU networks are often used for modeling sequential data. These models can capture temporal dependencies more explicitly, making them suitable for time series forecasting tasks.

4.4.4 Hybrid spatio-temporal models

More advanced approaches combine convolutional and recurrent components, enabling the model to learn both spatial and temporal dynamics simultaneously. Examples include ConvLSTM networks or transformer-based weather models. The objective of this study is to evaluate how well deep learning models can capture atmospheric patterns from reanalysis data and generate accurate short- to medium-range forecasts.

Implementation

5.1 Software Environment

The implementation of the weather forecasting pipeline was developed entirely in Python, using several scientific computing and machine learning libraries.

5.1.1 Python

Python is used as the main programming language for implementing the data pipeline, model training, evaluation procedures, and visualization of results. Python provides a flexible ecosystem for scientific computing and deep learning applications.

5.1.2 PyTorch

The deep learning model is implemented using PyTorch, a widely used machine learning framework that provides efficient GPU acceleration and automatic differentiation. PyTorch is used to define the neural network architecture, implement the training loop, compute the loss function, and perform inference with the trained model.

5.1.3 Numpy

The NumPy library is used for numerical computations and array manipulations. ERA5 data extracted from NetCDF files are converted into NumPy arrays before being transformed into tensors for model training.

5.1.4 Xarray

Meteorological data are stored in NetCDF format, which contains multidimensional arrays representing spatial and temporal variables. The xarray library is used to load these datasets, select variables, and manipulate atmospheric data efficiently before converting them into a format suitable for machine learning.

5.1.5 Matplotlib

The Matplotlib library is used for visualizing results. It is used to generate plots illustrating the training process and the performance of the forecasting model. In particular, graphs are produced to show how prediction errors (RMSE and MAE) evolve as the forecasting horizon increases.

5.1.6 Additional libraries

Other supporting libraries are also used in the pipeline:

- Torch utilities for dataset management and data loading
- JSON for saving experiment metrics and normalization parameters
- Zipfile for exporting experiment outputs and results

Together, these tools provide a complete environment for implementing the full forecasting workflow, from raw data processing to model evaluation and visualization.

5.2 Hardware Environment

The experiments were conducted using three different computing environments in order to evaluate the performance of the model under different hardware conditions. The Google Colab and local workstation environments were used exclusively for testing and debugging the code, while only the results obtained on the FinisTerra III supercomputer were retained for the final analysis presented in this work.

5.2.1 Google Colab GPU environment

Part of the early-stage experimentation was performed using Google Colab, which provides access to cloud-based GPUs for deep learning experiments. This environment allowed rapid testing and debugging of the training pipeline without requiring local GPU configuration. Google Colab provided a convenient platform for verifying the correctness of the PyTorch

implementation with GPU acceleration before scaling up to larger experiments. Results obtained in this environment were used solely for code validation and were not included in the final analysis.

5.2.2 Local workstation

Additional testing was performed on a local desktop computer equipped with an NVIDIA GeForce GTX 1660 Ti GPU. This GPU provided hardware acceleration for deep learning computations through CUDA support in PyTorch. Using a local GPU allowed faster debugging iterations compared to CPU execution and enabled full control over the experimental setup during the development phase. As with the Google Colab environment, this setup was used exclusively for code testing purposes, and the results produced here were not retained for analysis.

5.2.3 HPC supercomputer environment

In the context of the HPC (High Performance Computing) master’s program, the main experiments were carried out on the FinisTerra III supercomputer, granting access to both CPU and NVIDIA A100 GPU resources. This environment was used to compare execution times between CPU and GPU computations, allowing a clear assessment of the performance gains achieved through GPU acceleration. The most computationally intensive workloads were executed on the A100 GPUs, which provide significantly higher throughput and memory bandwidth than the GPUs used in the other two environments. As a result, all the results presented and analyzed in this work were obtained exclusively from experiments conducted on FinisTerra III, ensuring consistency and reliability across the reported performance metrics.

5.2.4 Hardware configuration summary

The main hardware resources used in this project include:

- GPU (cloud environment, testing only): Google Colab GPU
- GPU (local environment, testing only): NVIDIA GeForce GTX 1660 Ti
- CPU/GPU (HPC environment, analysis): FinisTerra III supercomputer with NVIDIA A100 GPUs
- Framework: PyTorch with CUDA support

The Google Colab and local workstation environments enabled rapid iteration and validation of the code during development, while all computationally demanding experiments and the

corresponding results discussed throughout this work were obtained on the FinisTerra III supercomputer, using its A100 GPUs to ensure both performance and reliability.

Experimental Results

6.1 Evaluation Metrics

To evaluate the performance of the forecasting model, several quantitative metrics are used. These metrics measure the difference between the temperature fields predicted by the neural network and the corresponding ground truth observations extracted from the ERA5 dataset. The main evaluation metrics used in this project are the following.

6.1.1 Mean Squared Error (MSE)

The Mean Squared Error (MSE) measures the average squared difference between predicted values and observed values. The formula can be written as:

$$\text{MSE} = (1 / N) \times \sum (\text{true value} - \text{predicted value})^2$$

True value represents the real temperature value from the dataset, predicted value represents the temperature predicted by the model and N is the total number of observations. This metric is used as the training loss function for the neural network. Minimizing the MSE encourages the model to produce predictions that are close to the real temperature fields.

6.1.2 Root Mean Squared Error (RMSE)

The Root Mean Squared Error (RMSE) is derived from the MSE and represents the square root of the mean squared error. The formula is: $\text{RMSE} = \sqrt{\text{MSE}}$ Unlike MSE, RMSE is expressed in the same unit as the predicted variable (Kelvin for temperature). This makes it easier to interpret the magnitude of prediction errors. RMSE is particularly useful for comparing model performance across different forecast horizons.

6.1.3 Mean Absolute Error (MAE)

In addition to RMSE, the Mean Absolute Error (MAE) is also used to evaluate the accuracy of the predictions. MAE measures the average absolute difference between predicted and true values:

$$\text{MAE} = \text{average of } |\text{true value} - \text{predicted value}|$$

Unlike MSE, MAE is less sensitive to large errors, which makes it useful for understanding the typical magnitude of prediction errors.

6.2 Visualization

Visual analysis of the results plays an important role in understanding the behavior of the forecasting model. Several visualizations are produced to evaluate both the training process and the quality of the generated forecasts.

6.2.1 Training process

The evolution of the training and validation losses is shown in Figure 6.1.

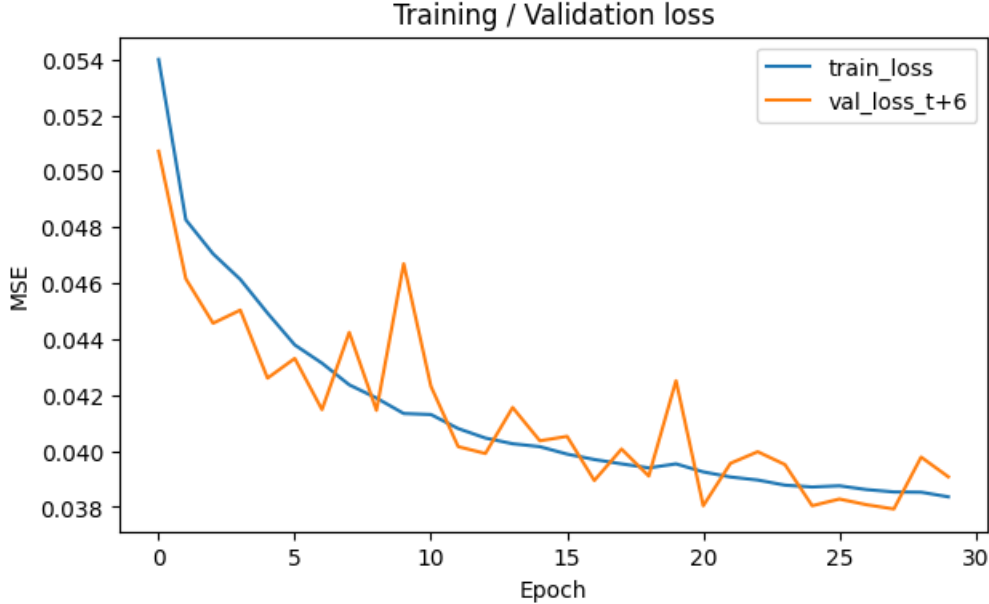


Figure 6.1: Learning curves of the U-Net model trained using the Mean Squared Error (MSE) loss function.

In Figure 6.1, the blue curve corresponds to the training loss, while the orange curve represents the validation loss for the prediction at $t + 6$ hours. Both curves exhibit a general de-

creasing trend throughout training, indicating that the neural network progressively learns to approximate the relationship between the atmospheric state at time t and the state at $t + 6$. Furthermore, the validation loss closely follows the training loss, suggesting that the model generalizes reasonably well to unseen data.

6.2.2 Example of temperature prediction

To qualitatively evaluate the model predictions, temperature maps generated by the model are compared with the corresponding ground truth observations, as illustrated in Figure 6.2.

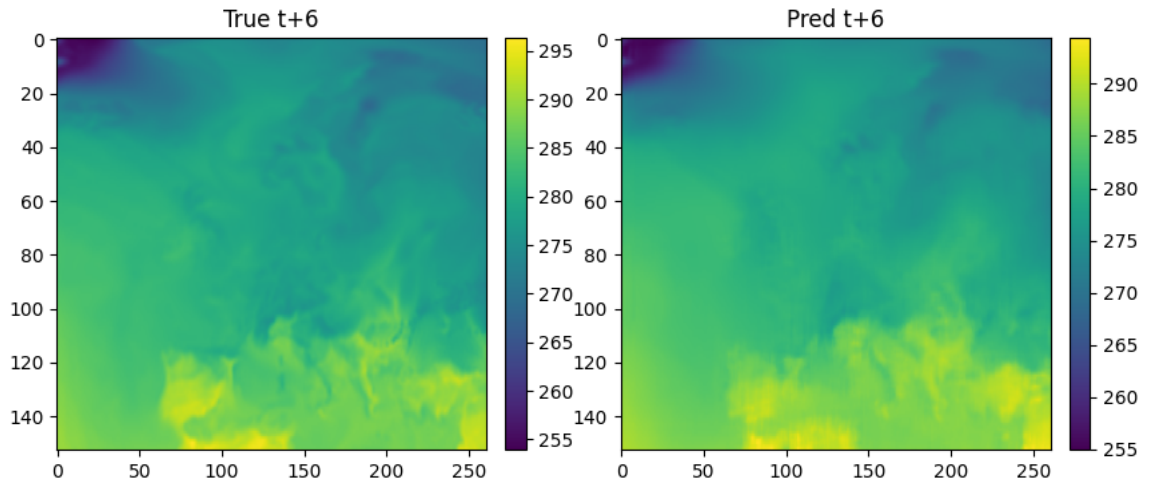


Figure 6.2: Spatial field comparison between ground truth (left) and model prediction (right) at lead time $t+6$.

Both maps in Figure 6.2 display the spatial distribution of temperature over the selected European region. The predicted map successfully reproduces the large-scale temperature patterns visible in the ground truth data. Major spatial structures, such as temperature gradients between northern and southern regions, are captured by the model. Although some local differences remain, the predicted field preserves the overall structure of the atmospheric temperature distribution, indicating that the model has learned meaningful spatial representations.

6.3 Forecast accuracy over multiple days

To analyze how prediction accuracy evolves as the forecast horizon increases, recursive forecasting is performed using the trained model. In this framework, the model first generates a prediction for the next time step (e.g., $t + 6$ hours), which is then fed back as input to predict the subsequent time step ($t + 12$ hours). This iterative process is repeated multiple times, allowing the model to produce forecasts several days ahead. While this approach enables

long-range predictions without requiring additional input data, it also introduces the accumulation of errors over time, as inaccuracies in earlier predictions can propagate and amplify in later steps.

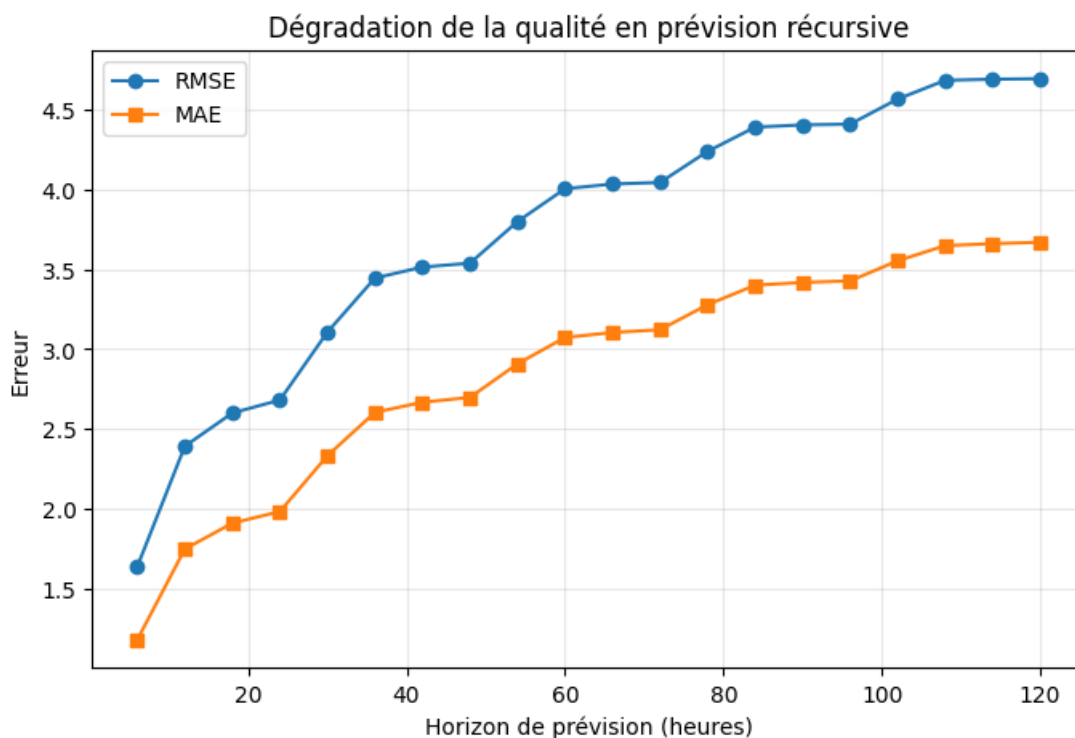


Figure 6.3: Evolution of RMSE and MAE as a function of forecast lead time.

Figure 6.3 illustrates the evolution of RMSE and MAE as a function of forecast lead time. Several observations can be made from this graph:

- Prediction errors increase as the forecast horizon becomes larger.
- The model achieves the lowest error for short-term forecasts (around 6 hours).
- As predictions extend to multiple days ahead, the error gradually increases due to the accumulation of uncertainty in recursive predictions.

This behavior, clearly visible in Figure 6.3, is consistent with typical weather forecasting systems, where short-term predictions are generally more accurate than long-range forecasts. Overall, the results demonstrate that the neural network is capable of producing reasonable short-term temperature forecasts and capturing important spatial structures in atmospheric data.

Discussion

The results obtained in this project show that deep learning models can successfully learn spatial patterns from meteorological data and generate short-term weather forecasts. The U-Net architecture used in this study demonstrated the ability to capture large-scale temperature structures and produce reasonable predictions for short forecast horizons. However, several aspects of the approach could be further explored to improve performance and better evaluate the effectiveness of the model. First, although the experiments mainly focused on a U-Net encoder-decoder architecture, other neural network architectures could also be investigated. For example, recurrent models such as LSTM or GRU networks could be used to better capture temporal dependencies in atmospheric data. Hybrid architectures such as ConvLSTM or transformer-based models could also be explored, as they combine spatial and temporal modeling capabilities. Testing several model families would provide a clearer understanding of which architectures are most suitable for short-term weather prediction. Another important aspect concerns the computational efficiency of the models. Future experiments could compare the training and inference times of different architectures. It is possible that lighter models with fewer parameters could achieve comparable prediction accuracy while requiring significantly less computational resources. Such comparisons would be particularly important for operational forecasting systems, where both accuracy and computational cost are critical factors. The computational experiments could also be extended by running the models on more powerful hardware. In this project, training was performed using GPUs available on Google Colab and a local NVIDIA GTX 1660 Ti. Running the training pipeline on more powerful accelerators, such as an NVIDIA A100 GPU, could allow the use of larger batch sizes, longer training runs, and larger datasets. This would make it possible to explore the effect of increasing the number of training epochs and determine whether additional training leads to improved accuracy. However, this would require careful monitoring in order to avoid overfitting, as excessive training can lead the model to memorize the training data rather than

generalize to unseen observations. Another possible improvement concerns the design of the model inputs. Instead of relying only on the temperature values at a single location or grid cell, the model could incorporate temperature information from geographically neighboring regions. Since atmospheric processes are spatially correlated, including surrounding temperature values could help the model better capture the evolution of local weather conditions and improve prediction accuracy. Further improvements could also involve modifying the hyperparameters of the U-Net architecture. For example, the number of encoder and decoder layers, the number of convolutional filters, or the types of layers used in the network could be adjusted to study their impact on model performance. Systematic hyperparameter tuning would allow a more comprehensive exploration of the architecture and potentially lead to better forecasting results. Another limitation of the current study is that only one meteorological variable, surface temperature, was used as input. In reality, atmospheric dynamics depend on multiple interacting variables such as wind speed, humidity, precipitation, and pressure. Future experiments could therefore incorporate additional meteorological variables into the model. For instance, variables such as rainfall or wind fields could provide additional information about atmospheric dynamics. It may also be possible to design a multi-variable model capable of predicting several atmospheric variables simultaneously, which could better reflect the complexity of real weather systems. In addition, the geographical scope of the experiments could be expanded. The current study focuses on a specific region over Europe, but it would be interesting to extend the experiments to other parts of the world. Training models on larger geographical regions or even global datasets could provide a better understanding of how well deep learning models generalize across different climates and atmospheric regimes. However, expanding the spatial domain would significantly increase the size of the datasets and the computational requirements for model training. As a result, careful consideration would need to be given to the available computational resources. Finally, it would be useful to compare the performance of the implemented model with more advanced AI-based weather forecasting systems. Recent models such as GraphCast, FourCastNet, or Pangu-Weather have demonstrated strong forecasting capabilities using large-scale datasets and advanced architectures. Although reproducing such models is beyond the scope of this project, comparing their reported performance with the results obtained here could help evaluate the relative efficiency of the proposed approach and identify potential directions for improvement. Overall, while the results obtained in this project are encouraging, further experimentation with different architectures, input configurations, meteorological variables, and computational setups would be necessary to fully assess the potential of deep learning methods for weather forecasting.

Conclusion

THIS project investigated the use of deep learning techniques for short-term weather forecasting using the ERA5 reanalysis dataset. A forecasting pipeline was implemented to process meteorological data, train neural network models, and evaluate their predictive performance. The results demonstrate that deep learning models can learn meaningful spatial representations of atmospheric variables and generate realistic temperature forecasts. The U-Net architecture used in this study was able to capture large-scale spatial structures in the temperature field and produce predictions that follow the overall distribution of the observed data. Evaluation metrics such as RMSE and MAE confirmed that the model performs well for short forecast horizons, although prediction errors increase progressively as the forecasting horizon extends further into the future. Despite these promising results, several limitations remain. The experiments were conducted using only a single meteorological variable (surface temperature), while real atmospheric systems involve complex interactions between many variables such as wind, humidity, and pressure. In addition, the experiments relied on relatively limited computational resources compared to the large-scale infrastructure used in operational forecasting systems. Future work could therefore focus on integrating additional atmospheric variables, experimenting with more advanced neural network architectures, and conducting larger-scale training experiments using more powerful hardware. Improvements in model design, hyperparameter optimization, and dataset diversity could significantly enhance forecasting accuracy. In conclusion, this project demonstrates the potential of deep learning approaches for modeling atmospheric dynamics and generating weather forecasts. Although these models are not yet a replacement for traditional numerical weather prediction systems, they represent a promising complementary approach that could contribute to the future development of data-driven weather forecasting methods.

Apéndice

Glosario de acrónimos

HPC *High Performance Computing.*

NWP *Numerical Weather Prediction.*

Glosario de terminos

Aceleración Ratio que relaciona el tiempo de ejecución de un proceso antes y después de una optimización aplicada sobre dicho proceso.

Bibliography

- [1] M. G. J. K. Małgorzata Murat, Małgorzata Murat, *Forecasting daily meteorological time series using ARIMA and regression models*, 1st ed. International Agrophysics, 2018.
- [2] M. Y. Radhika, *Atmospheric Temperature Prediction using Support Vector Machines*, 1st ed. International Journal of Computer Theory and Engineering, 2009.
- [3] R. C. Jonathan A. Weyn, Dale R. Durran, *IMPROVING DATA-DRIVEN GLOBAL WEATHER PREDICTION USING DEEP CONVOLUTIONAL NEURAL NETWORKS ON A CUBED SPHERE*, 1st ed., 2020.
- [4] D. I. N. D. L. Ilias Chatatidis, Georgios Tzanes and A. Stamou, *Short-Term Forecasting of Rainfall Using Sequentially Deep LSTM Networks*, 1st ed., 2023.
- [5] H. W. D.-Y. Y. W.-k. W. W.-c. W. Xingjian Shi, Zhourong Chen, *Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting*, 2nd ed., 2015.
- [6] S. M. Kevin Trebing, Tomasz Stanczyk, *SmaAt-UNet: Precipitation nowcasting using a small attention-UNet architecture*, 1st ed. Pattern Recognition Letters, 2021.
- [7] R. L. et al., *Learning skillful medium-range global weather forecasting*, 1st ed. Science, 2023.
- [8] Z. B. Bouallègue, M. C. A. Clare, L. Magnusson, E. Gascón, M. Maier-Gerber, M. Janoušek, M. Rodwell, F. Pinault, J. S. Dramsch, S. T. K. Lang, B. Raoult, F. Rabier, M. Chevalier, I. Sandu, P. Dueben, M. Chantry, and F. Pappenberger, *The Rise of Data-Driven Weather Forecasting: A First Statistical Assessment of Machine Learning-Based Weather Forecasts in an Operational-Like Context*, 1st ed. American Meteorological Society, 2024.

- [9] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar, *FourCastNet A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators*, 2022.